
Field validation checklist - the 0.7.x line

Human review of everything shipped 0.7.0-0.7.4, on real infrastructure - 2026-07-11

Saturday 11 July 2026

Contents

1	How to use this	1
2	1. Packages and CLI (SM139)	2
3	1b. Webserver glue (if testing on a plain host)	3
4	2. Domain onboarding (SM139 i4)	4
5	3. FastCGI pool (SM142)	5
6	4. Fleet operations (SM139 i3)	6
7	7. Content history + sync (SM085) - the new heart	7
8	Findings	8

1 How to use this

Work top to bottom on the ISP host; each item gives the command or click-path and the expected observation, so a failure localises fast. Sections are ordered so later items build on earlier ones (debs before pools before sync). Log anything surprising - wording, friction, and “that’s odd” reactions are wanted findings for the usability pass, not just hard failures. Versions under test: engine 0.7.5, debs 0.7.5-1.

2 1. Packages and CLI (SM139)

Install `sudo dpkg -i dist/lazysite-common_0.7.5-1_all.deb dist/lazysite-hestia_0.7.5-1_all.deb` - installs clean, no dependency errors (perl deps are all in place on this host). `man lazysite`, `man lazysite-hestia-domain render`.

CLI basics `lazysite version` - reports 0.7.4, channel edge. `lazysite sites` - lists nothing yet (or the registry you build below). `sudo lazysite provision --docroot /tmp/x` - REFUSES with the run-as-site-user message (the no-root principle; this refusal is a feature).

Host dependencies `lazysite check --dependencies` - all modules OK on this host, git listed in the environment section.

Instant demo `lazysite demo` (as your normal user) - fresh-installs a scratch site at `~/lazysite-demo` and serves `http://localhost:8080/` immediately; Ctrl-C stops it, `rm -rf ~/lazysite-demo` removes it. `sudo lazysite demo` REFUSES (a feature).

3 1b. Webserver glue (if testing on a plain host)

Skip on the Hestia host (Hestia owns its vhosts - section 2 covers it); on any plain Apache or nginx box, the 0.7.5 glue debs replace hand wiring.

Install + render `sudo dpkg -i dist/lazysite-apache_0.7.5-1_all.deb` (or `lazysite-nginx`). Provision a site as its user (`sudo -u <user> lazysite provision --docroot D --cgibin C --domain <d>`), then `sudo lazysite-apache-vhost add <d> --docroot D` (or `lazysite-nginx-vhost; --fcgi` for the pool pattern) - the vhost file appears in `sites-available`, the `a2enmod/enable/reload` steps are PRINTED not run. Run them; the site serves, `/manager` and `/login` work. On nginx check `fcgiwrap` is installed first.

Guard rails a second `add` without `--force` REFUSES; `remove <d>` deletes only the vhost file (`docroot` untouched); `man lazysite-apache-vhost / man lazysite-nginx-vhost render`. The wiring reference is in `/usr/share/doc/lazysite-apache/webserver-wiring.md` (same in `-nginx`).

4 2. Domain onboarding (SM139 i4)

New test domain create a scratch domain in Hestia (e.g. test074.), then: `sudo lazysite-hestia-domain add ispadmin test074.<domain> --channel edge` - one command; output shows the bounded root layout pass then provisioning as ispadmin; the site serves the starter immediately; `/etc/lazysite/sites.d/test074.<domain>` exists; `lazysite sites` lists it with owner/channel/policy.

Vhost template apply `lazysite-cgi` via the panel (v-change-web-domain-tpl) if the domain did not pick it up; `site + /manager + /login` all work.

Ownership correctness `sudo lazysite check --docroot .../test074.../public_html`
- clean report first time, NO -fix needed (the 0.6.5 era is over; a dirty first report here is a finding).

5 3. FastCGI pool (SM142)

Enable re-run onboarding with the pool: `sudo lazysite-hestia-domain add ... --fcgi` on a second scratch domain (or write `/etc/lazysite/pools/.conf` + `systemctl enable --now lazysite@<d>` on the first), apply the `lazysite-fcgi` template. `systemctl status lazysite@<d>` active; socket at `/run/lazysite/.sock`.

Feel the difference browse the pooled site - page loads should feel instant vs the CGI domain (cache hits ~0.4ms vs ~60ms). `ab -n 50` or repeated curl timing if you want numbers.

Manager still works /manager on the pooled site (manager traffic stays CGI by design - it must behave exactly as before). Edit a page; the edit appears on the pooled site immediately (cache invalidation reaches the pool's renders).

Worker hygiene `systemctl restart lazysite@<d>` mid-browsing - one failed/slow request at most, then normal.

6 4. Fleet operations (SM139 i3)

Policy + channel on the edge test domain: `sudo -u ispadmin lazysite upgrade --docroot ...` - “already 0.7.4” no-op. Set one real site’s conf to `update_policy: manual`, run `sudo lazysite upgrade --all` - the manual site is SKIPPED and logged, stable sites skip the edge payload, the edge test site upgrades (no-op). `--force-security` without a security-critical payload - REFUSES with the explanation (also a feature).

5. Sessions (SM141) Sign in from two browsers (or one + private window). `/manager/sessions` under Access: both sessions listed with IP/device, yours badged “this session”. Sign out the OTHER one - its next click bounces to `/login`; yours keeps working. Sign back in, then “Sign out everywhere” for your user from one of them - BOTH die. Audit page shows the revocations. Wording check: is the page self-explanatory to someone who has never heard “revocation list”?

6. Domain aliases (SM110 + SM134) On a test site’s conf: `alias_hosts: alias074.<domain> + alias.alias074.<domain>.site_name: The Other Brand` (+ theme if two installed). Point the extra host at the same docroot (Hestia domain alias or hosts-file trick). The alias host shows its own name/theme; the primary is untouched; browse both alternately - no bleed either way (this is the cache-isolation test). Edit a page; BOTH hosts show the edit on next load. `aliases_temp: /offer` front matter on a page -> `/offer 302s` (`curl -l`); `aliases: entry 301s`. Files page shows the Aliases card with badges.

7 7. Content history + sync (SM085) - the new heart

Enable Backups page -> Content history card -> Enable. Status flips to enabled with the adoption commit counted. `sudo lazysite check --docroot ... -git` probes ok (and confirm: NO /.git under public_html; the repo is at lazysite/git/).

History + step-through edit a page three times with distinct content. Files -> the page -> History: three entries, your user attributed, sensible messages. View an old version (raw content correct); Diff (readable); Restore the oldest - page serves the old content, the RESTORE itself tops the history, audit shows it, cache refreshed.

The exclusion proof (worth doing once by hand) `sudo -u ispadmin git --git-dir .../lazysite/git --work-tree .../public_html ls-files | grep -E 'auth/|forms/|notify-xmpp|git-sync'` - MUST return nothing. This is the “safe to push” guarantee.

Remote sync create a private empty repo on your git host (Forgejo/other). Plugin Manager -> enable Git sync; Plugin Config -> remote URL + token + Test connection (“reachable, ready”). PUSH - “Pushed N changes”; the remote shows the site content and ONLY safe content (spot-check: no auth/, no conf tokens). Edit a file directly on the remote (web UI), PULL - “1 change applied”, the page updates on-site. Now edit the SAME page locally AND remotely differently, PULL - it stops: “changed in both places” + Keep mine / Take theirs buttons; pick one; verify the result and that a safety snapshot appeared under Backups. Wording check throughout: any git jargon that slipped through is a finding.

8. Analytics + recovery (0.6.x regression sweep) /manager/stats on a fresh 0.7.4 site - data appears without ANY server setup (first-party). Bad-URL card lists probe blocks if any. Break something on purpose on a scratch site (`chmod 0700 lazysite/manager`) and run `sudo lazysite check --docroot ... --fix` - fixes applied AND the printed report reflects the post-fix state (says so explicitly).

9. Upgrade-in-anger drill (when 0.7.5+ exists) `sudo lazysite upgrade --all` across the fleet as the routine roll; time it; note any site needing manual attention - the target is zero.

8 Findings

Record here (or straight into reported-issues.md / the backlog):

#	Item	Finding	Severity
1			